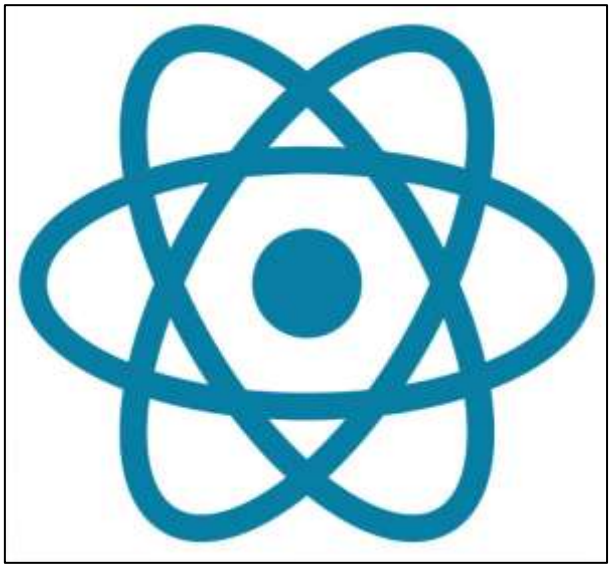
REACT Y COMPONENTES

React

React es una biblioteca de JavaScript utilizada para construir interfaces de usuario (UI) interactivas y dinámicas, especialmente en aplicaciones web modernas. Fue desarrollada inicialmente por Facebook, actualmente Meta, y permite construir interfaces a partir de componentes reutilizables.

Por ejemplo, si desarrollamos un sistema universitario, React puede utilizarse para crear la pantalla de inicio de sesión, el formulario de matrícula, la lista de estudiantes, el menú de navegación y el panel de calificaciones.



1. ¿Para qué sirve React?

React facilita el desarrollo de interfaces que cambian según las acciones del usuario o los datos recibidos del servidor.

Entre sus principales aplicaciones se encuentran:

- Formularios de registro, matrícula e inicio de sesión.
- Tablas y listas de datos que se actualizan dinámicamente.
- Paneles de control con indicadores y gráficos.
- Carritos de compra y catálogos de productos.
- Aplicaciones web que consumen APIs.
- Interfaces con pestañas, ventanas modales, filtros y búsquedas.

Una de sus características más importantes es que permite dividir una interfaz compleja en componentes pequeños, independientes y reutilizables.

2. ¿Cómo funciona React?

Imaginemos que una página web universitaria tiene tres elementos principales:



El proceso general es el siguiente:

a) Se definen componentes: por ejemplo, Menu, FormularioMatricula y ListaEstudiantes.
b) Se reciben datos: mediante propiedades llamadas props o a través del estado del componente.
c) El usuario interactúa: escribe en un campo, pulsa un botón o selecciona una opción.
d) Se actualiza el estado: React detecta el cambio y actualiza las partes necesarias de la interfaz.

React utiliza un mecanismo de representación conocido como Virtual DOM para ayudar a gestionar las actualizaciones de la interfaz. Sin embargo, su principal ventaja no consiste únicamente en ese mecanismo, sino en organizar la interfaz mediante componentes y un modelo declarativo de programación.

3. Ejemplo básico de un componente React

Un componente es una función de JavaScript que devuelve una descripción de la interfaz que React debe mostrar. Frecuentemente se utiliza JSX, una sintaxis que permite escribir estructuras parecidas a HTML dentro de JavaScript.

JavaScript

```
function Saludo() {
  return (
    <div>
      <h2>Bienvenido a la aplicación</h2>
      <p>Este es mi primer componente React.</p>
    </div>
  );
}

export default Saludo;
```

En este ejemplo:

- function Saludo() define el componente.
- return devuelve los elementos de la interfaz.
- JSX permite combinar etiquetas con expresiones de JavaScript.
- export default Saludo permite importar el componente desde otro archivo.

Para mostrarlo desde otro componente, se puede escribir <Saludo />.

4. Ejemplo de React aplicado a un formulario

Como continuación de los ejemplos de formularios que hemos revisado, veamos cómo implementar un formulario de registro de estudiantes con React.

JavaScript

```
import { useState } from "react";

function FormularioEstudiante() {
  const [nombre, setNombre] = useState("");
  const [correo, setCorreo] = useState("");
  const [mensaje, setMensaje] = useState("");

  function registrar(event) {
    event.preventDefault();

    if (!nombre.trim() || !correo.trim()) {
      setMensaje("Complete todos los campos.");
      return;
    }

    setMensaje(`Estudiante registrado: ${nombre}`);
  }
```

```
return (
  <form onSubmit={registrar}>
    <h2>Registro de estudiantes</h2>

    <label htmlFor="nombre">Nombre:</label>
    <input
      id="nombre"
      value={nombre}
      onChange={(event) => setNombre(event.target.value)}
      required
    />

    <br /><br />

    <label htmlFor="correo">Correo:</label>
    <input
      id="correo"
      type="email"
      value={correo}
      onChange={(event) => setCorreo(event.target.value)}
      required
```

```
    />

    <br /><br />

    <button type="submit">Registrar</button>

    <p role="status">{mensaje}</p>
  </form>
);
}

export default FormularioEstudiante;
```

Este componente muestra los conceptos fundamentales de React:

Concepto	Explicación
`useState`	Permite almacenar datos que pueden cambiar.
`nombre` y `correo`	Representan el estado de los campos.
`setNombre()`	Actualiza el nombre y provoca la actualización correspondiente de la interfaz.
`onChange`	Detecta los cambios en los campos.
`onSubmit`	Gestiona el envío del formulario.
`event.preventDefault()`	Evita el envío tradicional del navegador.
JSX	Describe los campos, el botón y el mensaje.

Importante: este ejemplo solo simula el registro en el frontend. Para guardar estudiantes de verdad, se necesita conectar el componente con una API o un backend.

5. Diferencias entre JavaScript tradicional y React

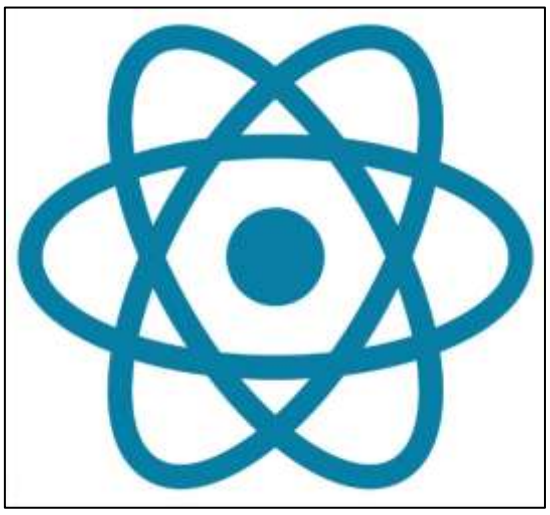
Aspecto	JavaScript tradicional	React
Construcción de interfaces	Manipulación directa del DOM cuando es necesario.	Interfaces declaradas mediante componentes.
Actualización de datos	El programador suele actualizar manualmente los elementos del DOM.	La interfaz se actualiza a partir del estado y las propiedades.
Reutilización	Puede lograrse mediante funciones y módulos.	Se organiza habitualmente en componentes reutilizables.
Formularios	Se gestionan con eventos y elementos HTML.	Pueden gestionarse mediante estado y eventos de React.
Organización de aplicaciones grandes	Depende de la arquitectura diseñada.	Ofrece un modelo centrado en componentes.

React no reemplaza a HTML, CSS ni JavaScript: utiliza JavaScript y JSX para construir la interfaz, y puede combinarse con CSS para definir su presentación.

6. ¿Qué tecnologías se relacionan con React?

React

Biblioteca para construir interfaces mediante componentes.



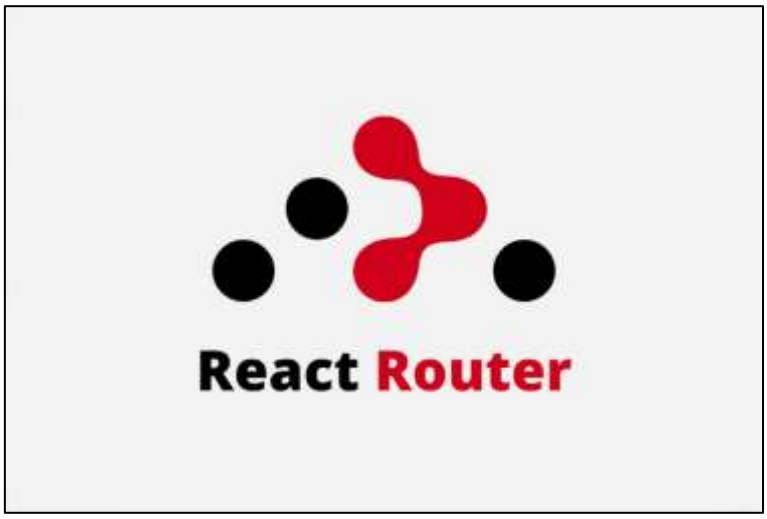
Vite

Permite crear y ejecutar un entorno de desarrollo moderno para proyectos React.



React Router

Facilita la navegación entre vistas y rutas de una aplicación



También se suele combinar React con herramientas para consumir APIs, gestionar formularios, aplicar estilos y realizar pruebas. React por sí solo no proporciona necesariamente todas las funciones de una aplicación completa.

7. ¿Cómo comenzar a aprender React?

Si se conoce HTML, CSS y JavaScript, se recomienda:

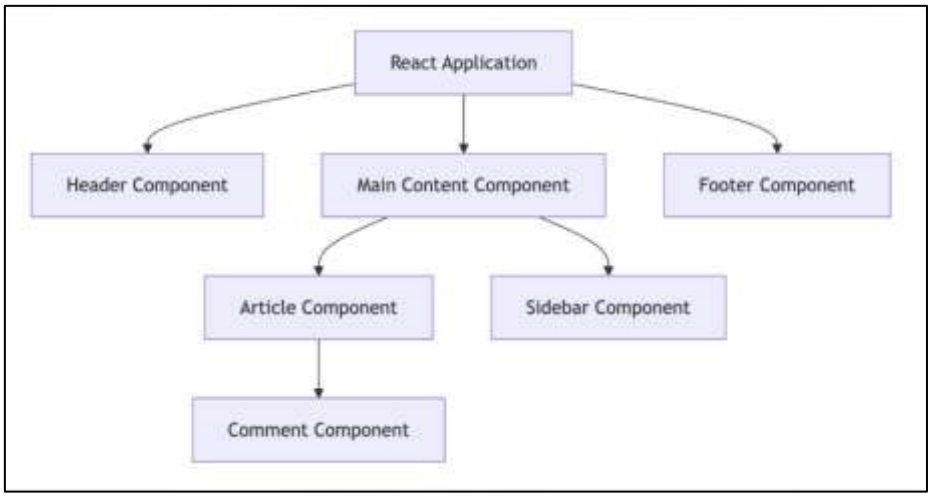
a) Componentes y JSX.
b) Propiedades (props).
c) Estado y useState.
d) Eventos y formularios.

e) Renderizado de listas y uso de map().
f) Renderizado condicional.
g) Efectos con useEffect.
h) Consumo de APIs mediante fetch().
i) Enrutamiento y organización de componentes.
j) Desarrollo de una aplicación integrada con backend.

Características de los componentes React.

En React, los componentes son bloques reutilizables de una interfaz de usuario que permiten dividir una aplicación web en partes pequeñas, organizadas y fáciles de mantener.

Por ejemplo, en un sistema de gestión universitaria podemos tener componentes para el menú principal, el formulario de matrícula, la lista de estudiantes, las calificaciones y las notificaciones.



La principal ventaja es que cada componente puede tener una responsabilidad específica y combinarse con otros para construir una aplicación completa.

1. Principales características

a) Modularidad

Una interfaz se divide en componentes más pequeños. Cada uno se encarga de una parte de la aplicación.

Ejemplo: MenuPrincipal, FormularioMatricula y ListaEstudiantes.

b) Reutilización

Un componente puede utilizarse varias veces sin tener que escribir nuevamente toda su estructura.

Ejemplo: utilizar el mismo componente Boton para registrar, actualizar o eliminar información, con las adaptaciones necesarias.

c) Personalización mediante props

Las props son propiedades que permiten enviar información de un componente padre a uno hijo.

Ejemplo: pasar el nombre y el código de un estudiante a una tarjeta de presentación.

d) Actualización mediante estado

Un componente puede mantener información que cambia durante su ejecución. Cuando cambia su estado, React actualiza la interfaz que depende de él.

Ejemplo: mostrar el número de estudiantes registrados o el texto introducido en un campo.

e) Composición

Los componentes pueden contener otros componentes. Así se construyen interfaces complejas a partir de elementos sencillos.

Ejemplo: una página de matrícula puede combinar encabezado, menú, formulario y pie de página.

f) Declaratividad

El desarrollador describe cómo debe verse la interfaz según los datos y el estado actuales. React se encarga de actualizar el DOM cuando corresponde.

Ejemplo: indicar que se muestre «Matrícula registrada» cuando la operación haya terminado correctamente.

g) Uso de JSX

Los componentes suelen utilizar JSX, una extensión de sintaxis que permite describir elementos similares a HTML dentro de JavaScript.

Esto facilita leer y organizar la estructura de la interfaz.

2. ¿Cómo se define un componente React?

Actualmente, la forma habitual de crear componentes es mediante funciones de JavaScript que devuelven JSX.

JavaScript

```
function Saludo() {
  return (
    <section>
      <h2>Bienvenido al sistema</h2>
      <p>Gestión académica universitaria</p>
    </section>
  );
}

export default Saludo;
```

En este ejemplo:

- Saludo es el nombre del componente. Los nombres de componentes deben comenzar con mayúscula.
- function declara una función de JavaScript.
- return devuelve el contenido que React debe representar.
- JSX describe los elementos de la interfaz.
- export default permite importar el componente desde otro archivo.

Para utilizarlo dentro de otro componente, se escribe:

JavaScript

```
function Aplicacion() {
  return (
    <main>
      <Saludo />
    </main>
  );
}
```

La etiqueta <Saludo /> representa una instancia del componente Saludo.

3. Propiedades (props): recibir información

Las propiedades permiten configurar un componente desde el exterior. Son especialmente útiles para reutilizar la misma estructura con datos diferentes.

JavaScript

```
function TarjetaEstudiante({ nombre, carrera }) {
  return (
    <article>
      <h3>{nombre}</h3>
      <p>Carrera: {carrera}</p>
    </article>
  );
}

function Aplicacion() {
  return (
    <div>
      <TarjetaEstudiante
        nombre="Ana Pérez"
        carrera="Ingeniería de Sistemas"
      />

      <TarjetaEstudiante
        nombre="Luis Torres"
        carrera="Ingeniería de Software"
      />
    </div>
  );
}
```

El mismo componente muestra dos estudiantes diferentes.

Características de las props:

- Se reciben desde el componente padre.
- Permiten personalizar el contenido.
- Son de solo lectura para el componente que las recibe: no deben modificarse directamente.
- Pueden contener cadenas, números, objetos, arreglos, funciones u otros elementos de React.

4. Estado (state): gestionar cambios

Cuando una interfaz debe reaccionar a las acciones del usuario, se puede utilizar el estado mediante el Hook useState.

El siguiente ejemplo implementa un contador de estudiantes registrados.

JavaScript

```jsx
import { useState } from "react";

function ContadorEstudiantes() {
  const [cantidad, setCantidad] = useState(0);

  return (
    <section>
      <h3>Estudiantes registrados: {cantidad}</h3>

      <button onClick={() => setCantidad(cantidad + 1)}>
        Registrar estudiante
      </button>
    </section>
  );
}

export default ContadorEstudiantes;
```

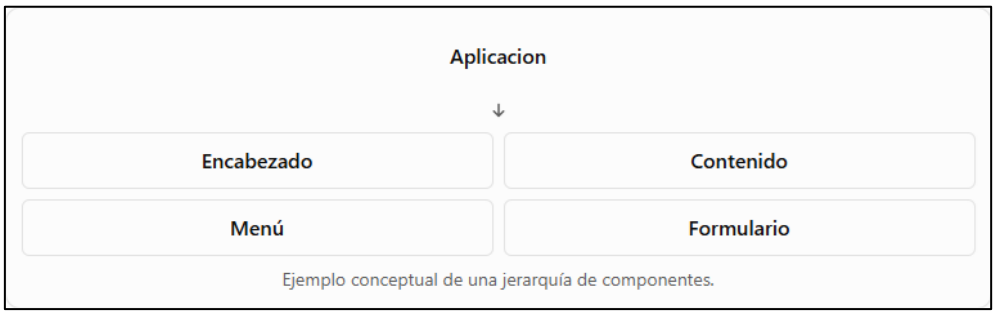
Al pulsar el botón, setCantidad actualiza el estado y React vuelve a representar el componente para reflejar el nuevo valor.

Es importante distinguir:

Props	State
Datos recibidos desde el componente padre.	Datos administrados por el propio componente o su lógica de estado.
El componente receptor no debe modificarlas.	Se actualiza mediante la función correspondiente, como `setCantidad`.
Permiten configurar componentes reutilizables.	Permite representar cambios e interacciones.

5. Composición y jerarquía de componentes

Los componentes pueden organizarse en una estructura jerárquica. Por ejemplo:



Ejemplo conceptual de una jerarquía de componentes.

Esta organización permite desarrollar, probar y mantener cada parte con mayor independencia. Además, facilita que varios integrantes de un equipo trabajen en distintos componentes de una misma aplicación.

REACT Y COMPONENTES

6. Otras características importantes

- Eventos: los componentes responden a acciones como onClick, onChange y onSubmit.
- Renderizado condicional: pueden mostrar contenido diferente según el estado o las propiedades.
- Renderizado de listas: pueden generar elementos de una colección mediante map(), utilizando una propiedad key estable y única entre los elementos hermanos.
- Separación de responsabilidades: conviene que cada componente tenga una función clara.
- Mantenibilidad y pruebas: los componentes bien diseñados pueden verificarse y modificarse con mayor facilidad.
- Efectos secundarios: cuando se necesita sincronizar un componente con sistemas externos, como una suscripción o ciertos servicios, se puede utilizar useEffect; no debe usarse automáticamente para cualquier cálculo o cambio de estado.

Conclusión

Las características fundamentales de los componentes React son la modularidad, reutilización, composición, uso de props, gestión del estado y actualización declarativa de la interfaz. Comprender estos conceptos permite pasar de páginas construidas mediante manipulación directa del DOM a aplicaciones web organizadas en componentes que responden a los datos y a las acciones del usuario.

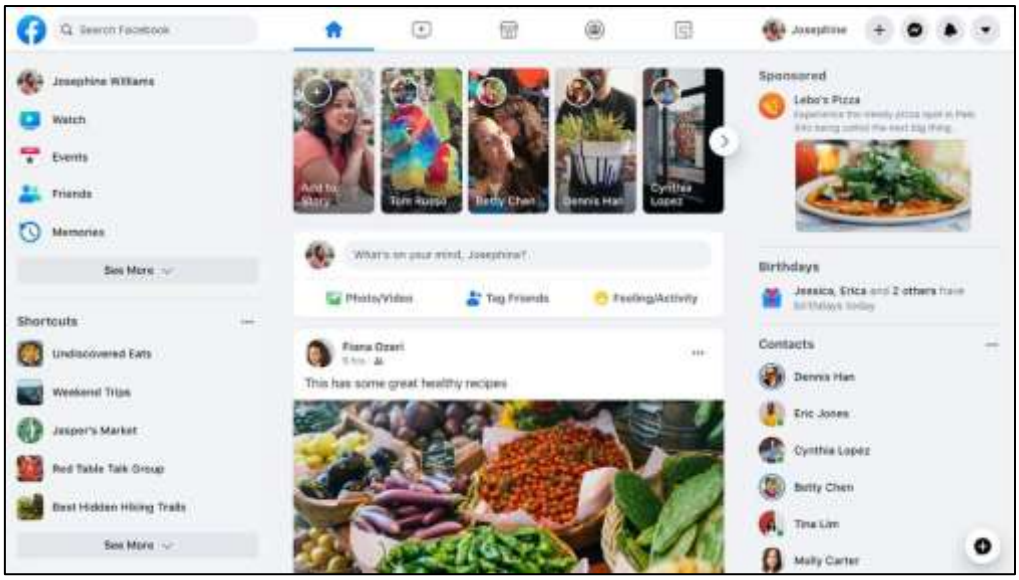
Ejemplo de aplicaciones con React.

React permite desarrollar desde pequeñas interfaces interactivas hasta aplicaciones web completas. Se utiliza principalmente para construir el frontend, es decir, la parte de la aplicación con la que interactúa el usuario. Para gestionar bases de datos, autenticación y lógica de negocio, normalmente se integra con un backend o con servicios externos.

A) Facebook (Red social)

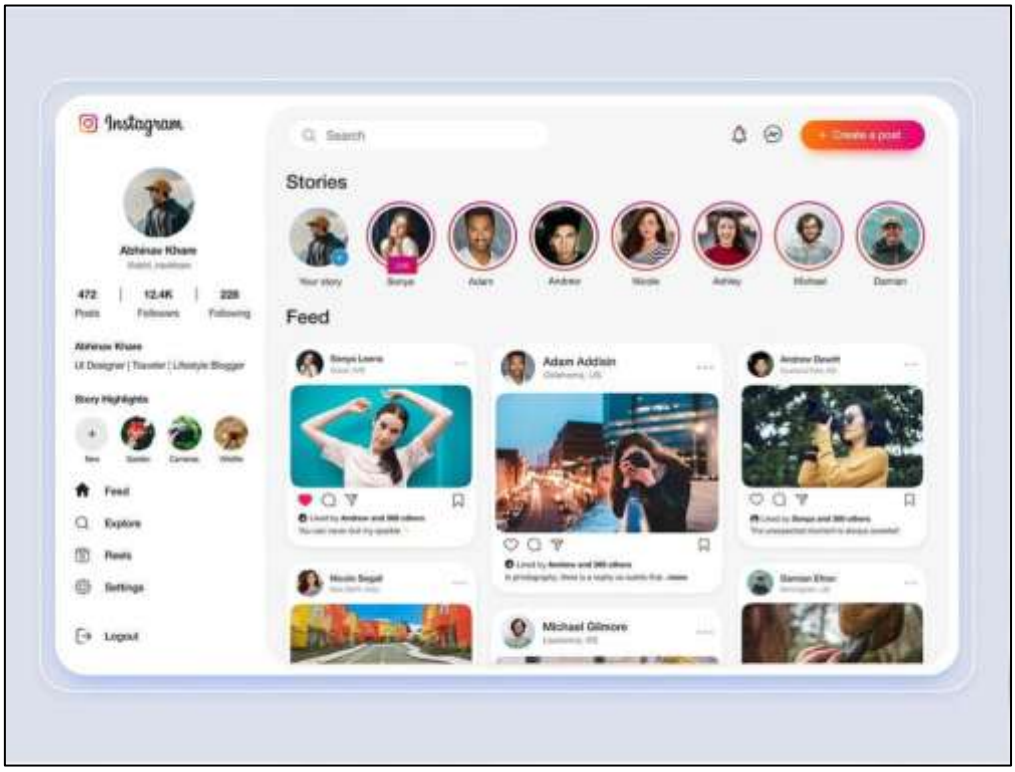
Permite visualizar publicaciones, comentar, reaccionar, explorar perfiles y actualizar el contenido de forma dinámica. React fue creado originalmente en Facebook y se utiliza en partes de su ecosistema de interfaces.

REACT Y COMPONENTES



B) Instagram (Red social)

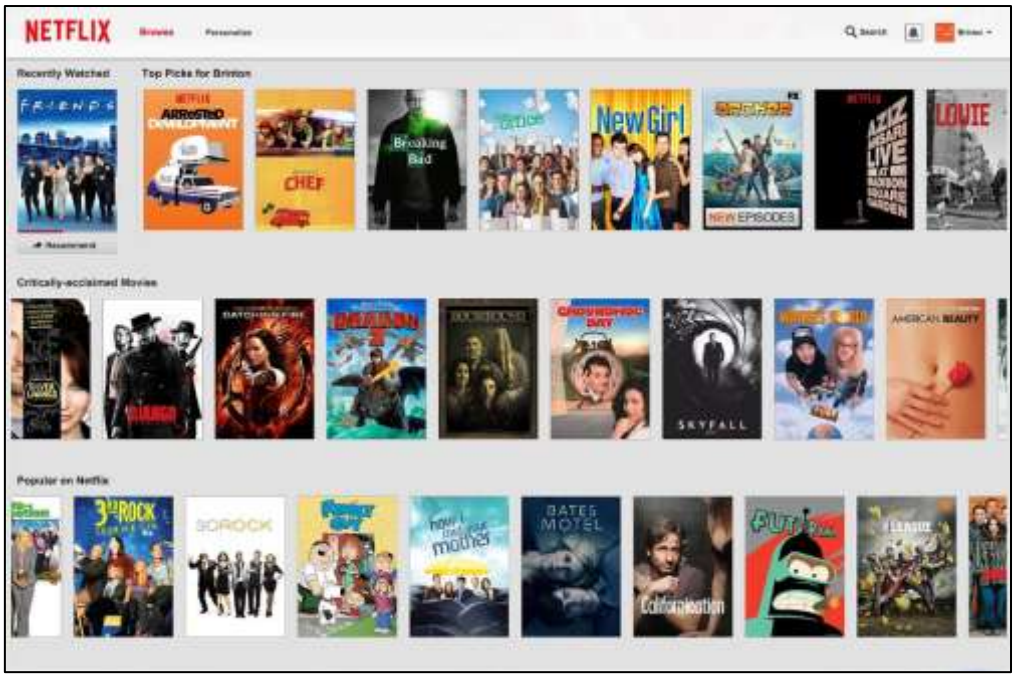
Su interfaz incluye publicaciones, perfiles, comentarios, imágenes y navegación entre secciones. Es un ejemplo de una experiencia de usuario que necesita gestionar contenido e interacciones frecuentes.



C) Netflix (Entretenimiento)

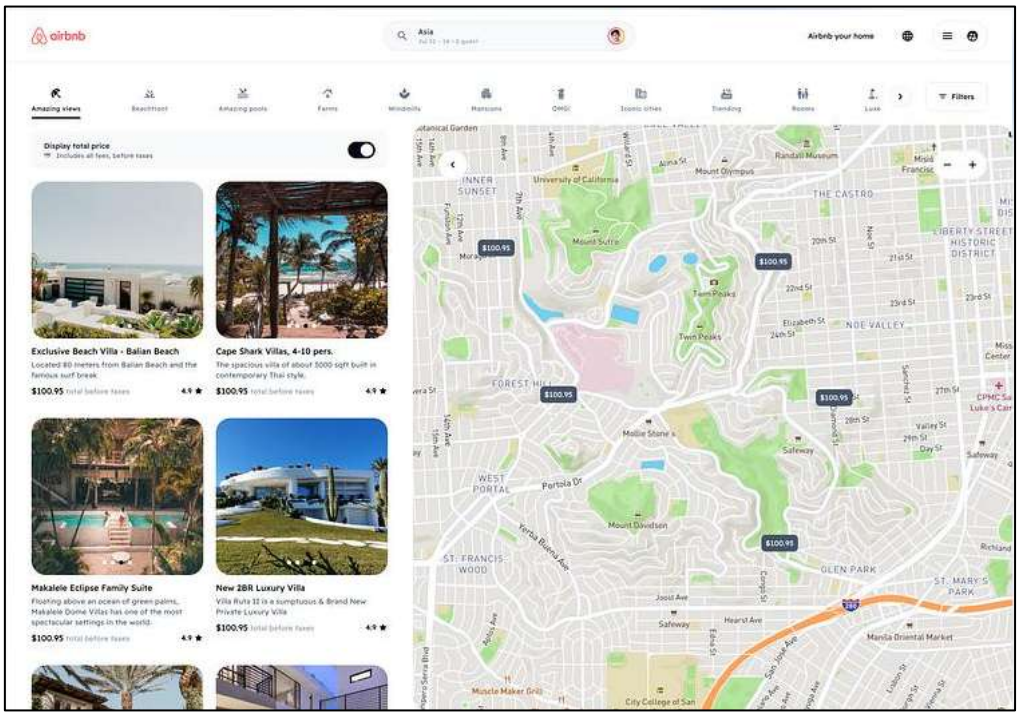
Presenta catálogos, categorías, búsquedas y recomendaciones. React puede utilizarse para construir partes de interfaces de este tipo, aunque la arquitectura de cada plataforma puede incluir otras tecnologías.

REACT Y COMPONENTES



D) Airbnb (Comercio y servicios)

Sus interfaces de búsqueda, filtros, tarjetas de alojamientos y procesos de reserva ilustran cómo se pueden combinar componentes reutilizables y estados interactivos.



Estas plataformas son referencias de productos digitales con interfaces complejas; no significa que todas sus páginas estén construidas exclusivamente con React ni que todas sus versiones utilicen la misma tecnología.

2. Ejemplo funcional: una pequeña aplicación de tareas

Este ejemplo permite agregar tareas, marcarlas como completadas y eliminarlas. Es una aplicación sencilla que demuestra el uso de componentes, estado y eventos en React.

JavaScript

```
import { useState } from "react";

function App() {
  const [texto, setTexto] = useState("");
  const [tareas, setTareas] = useState([]);

  function agregarTarea(event) {
    event.preventDefault();

    if (!texto.trim()) return;

    setTareas([
      ...tareas,
      {
        id: crypto.randomUUID(),
        texto: texto.trim(),
        completada: false
      }
    ]);

  setTexto("");
}

function cambiarEstado(id) {
  setTareas(tareas.map(tarea =>
    tarea.id === id
      ? { ...tarea, completada: !tarea.completada }
      : tarea
  ));
}

function eliminarTarea(id) {
  setTareas(tareas.filter(tarea => tarea.id !== id));
}

return (
  <main>
    <h1>Gestor de tareas</h1>

    <form onSubmit={agregarTarea}>
      <input
```

```
    value={texto}
    onChange={event => setTexto(event.target.value)}
    placeholder="Escriba una tarea"
    aria-label="Nueva tarea"
    required
  />
  <button type="submit">Agregar</button>
</form>

<ul>
  {tareas.map(tarea => (
    <li key={tarea.id}>
      <label>
        <input
          type="checkbox"
          checked={tarea.completada}
          onChange={() => cambiarEstado(tarea.id)}
        />
        <span>
          {tarea.completada
            ? `✓ ${tarea.texto}`
                : tarea.texto}
          </span>
        </label>

        <button
          type="button"
          onClick={() => eliminarTarea(tarea.id)}
        >
          Eliminar
        </button>
      </li>
    ))}
  </ul>

  <p>Total de tareas: {tareas.length}</p>
  <p>
    Completadas: {tareas.filter(t => t.completada).length}
  </p>
</main>
  );
}
```

```
export default App;
```

¿Qué conceptos de React se aplican?

- useState: almacena el texto y la lista de tareas.
- onChange: actualiza el texto mientras se escribe.
- onSubmit: procesa el formulario.

- map(): representa cada tarea en la interfaz.
- filter(): permite eliminar tareas y contar las completadas.
- key: identifica cada elemento de la lista.
- El operador ...: permite crear un nuevo arreglo sin modificar directamente el estado anterior.

Este ejemplo conserva los datos solo mientras la aplicación permanece activa; para mantenerlos después de recargar la página habría que incorporar almacenamiento local o un backend.

Node.js como entorno de desarrollo.

Node.js es un entorno de ejecución de JavaScript que permite ejecutar código JavaScript fuera del navegador, por ejemplo, en un servidor, en la terminal de comandos o como parte de herramientas de desarrollo.

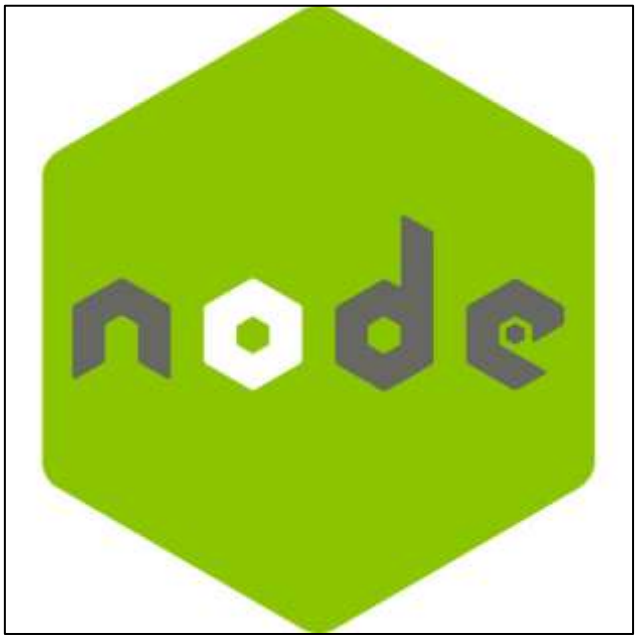
Tradicionalmente, JavaScript se asociaba principalmente con la interacción dentro de las páginas web. Con Node.js, los desarrolladores también pueden utilizar JavaScript para crear servidores, APIs, aplicaciones de red y herramientas que automatizan tareas de desarrollo.

Una distinción importante es que Node.js no es un lenguaje de programación ni un framework: es un entorno que permite ejecutar JavaScript utilizando, entre otros componentes, el motor V8.

1. ¿Para qué sirve Node.js?

Desarrollar servidores web

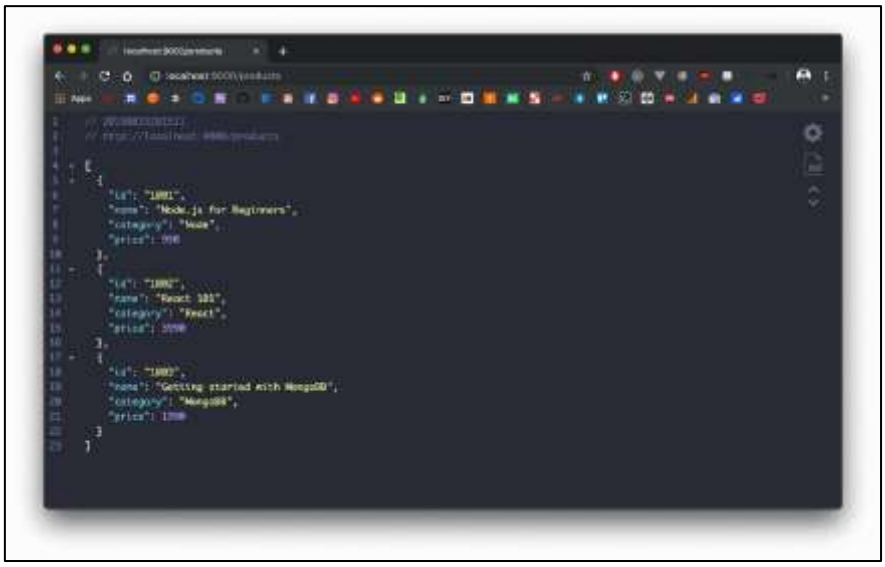
Recibir solicitudes HTTP, procesarlas y devolver respuestas al navegador.



REACT Y COMPONENTES

Construir APIs

Crear servicios para que aplicaciones React, móviles u otros sistemas consulten o actualicen información.



Gestionar herramientas y dependencias

Utilizar paquetes, automatizar tareas y ejecutar herramientas de desarrollo web, incluidas las empleadas en proyectos React.

```
→  calculator-cli-tool npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

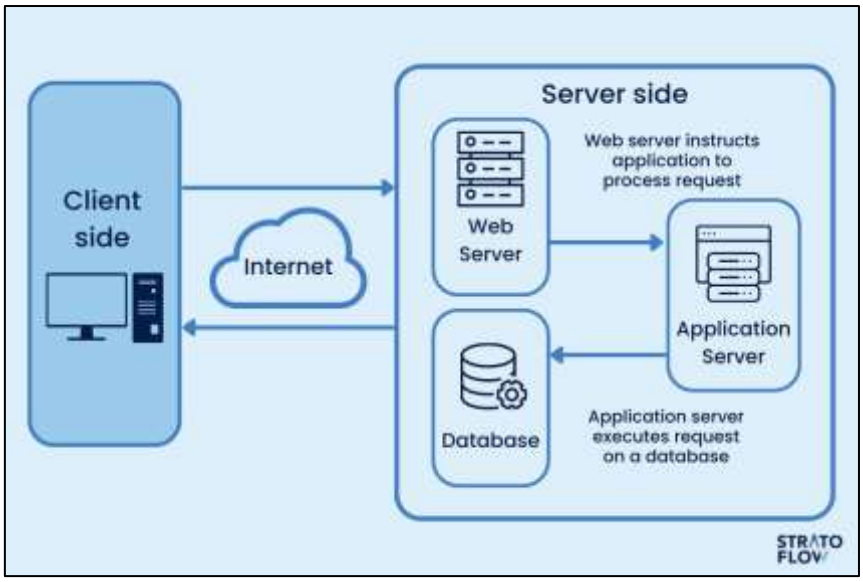
Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (calculator-cli-tool)
version: (1.0.0)
description: A CLI tool used for calculations
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to /Users/I1333/Desktop/self-study/calculator-cli-tool/package.json:

{
  "name": "calculator-cli-tool",
  "version": "1.0.0",
  "description": "A CLI tool used for calculations",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}


Is this OK? (yes) yes

→  calculator-cli-tool
```

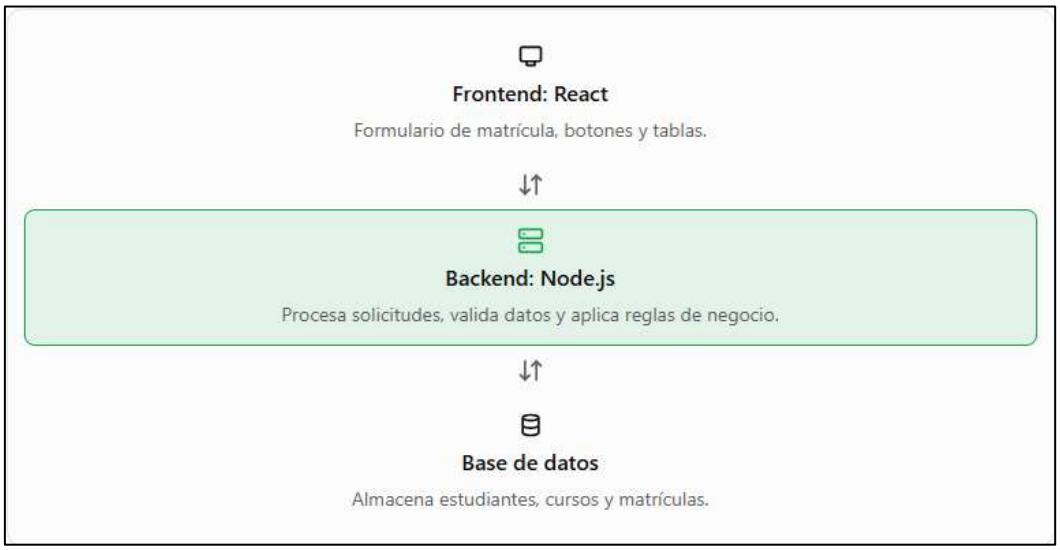


Conectar con bases de datos y servicios

Integrarse con sistemas SQL, NoSQL, servicios externos y sistemas de autenticación mediante bibliotecas apropiadas.

2. ¿Cómo funciona Node.js dentro de una aplicación web?

Consideremos un sistema universitario desarrollado con React y Node.js.



El flujo sería el siguiente:

a) Un estudiante completa el formulario en React.
b) El frontend envía los datos a una API mediante fetch().
c) Node.js recibe la solicitud HTTP.
d) El backend valida la información y verifica las reglas de negocio.
e) Si corresponde, consulta o actualiza la base de datos.
f) Node.js devuelve una respuesta que React presenta al usuario.

Node.js puede funcionar como backend de esta arquitectura, pero también puede utilizarse sin React o junto con otros frameworks frontend.

3. Principales características de Node.js

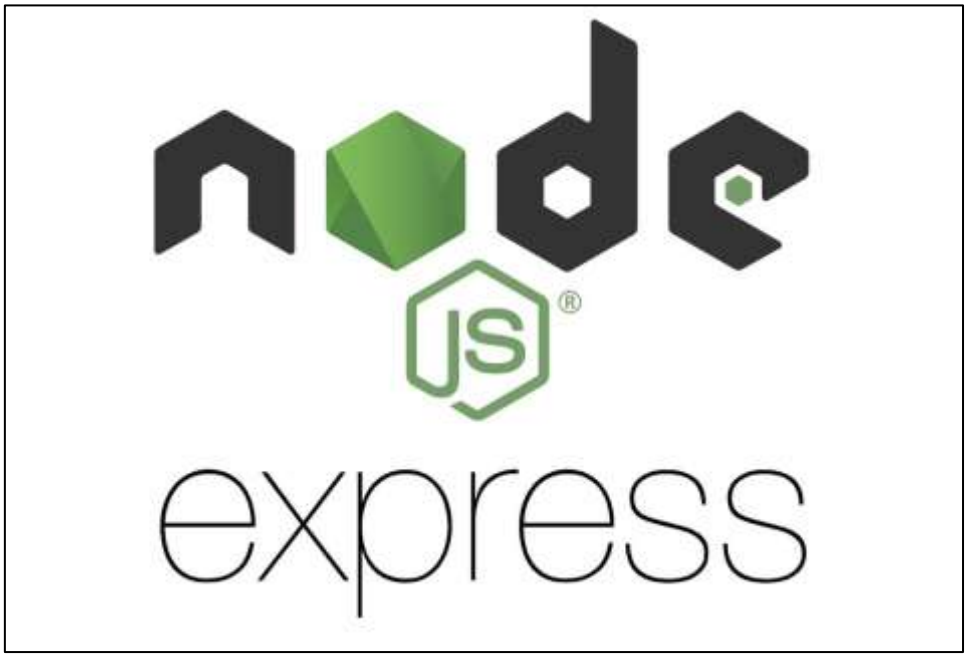
Característica	Explicación
Ejecución de JavaScript	Ejecuta código JavaScript fuera del navegador.
Motor V8	Utiliza el motor de JavaScript desarrollado por Google.
Entrada y salida asíncronas	Permite gestionar operaciones de archivos, red y otras tareas sin bloquear innecesariamente el flujo principal.
Modelo orientado a eventos	Utiliza un ciclo de eventos (*event loop*) para coordinar operaciones asíncronas.
Ecosistema npm	Permite instalar y administrar bibliotecas y herramientas.
Multiplataforma	Puede ejecutarse en Windows, Linux y macOS.
Módulos	Permite organizar el código en archivos y módulos reutilizables.
Desarrollo de servidores	Incluye módulos para trabajar con HTTP, archivos, procesos y comunicaciones de red.

Node.js es especialmente adecuado para aplicaciones que manejan muchas operaciones de entrada y salida concurrentes. No obstante, las tareas intensivas de cálculo pueden bloquear el hilo principal si no se gestionan adecuadamente.

4. Node.js y Express

Express

Es un framework web para Node.js que facilita definir rutas, gestionar solicitudes HTTP, utilizar middleware y construir APIs.



5. ¿Qué es npm y qué relación tiene con Node.js?

npm es el gestor de paquetes que habitualmente acompaña a Node.js. Permite instalar bibliotecas, administrar dependencias y ejecutar scripts de un proyecto.

Por ejemplo:

Comando	Función
`npm init -y`	Crea un archivo inicial `package.json`.
`npm install express`	Instala Express como dependencia.
`npm install`	Instala las dependencias declaradas en el proyecto.
`npm run dev`	Ejecuta el script `dev`, si está definido en `package.json`.
`npm run start`	Ejecuta el script `start`, si está definido.

En un proyecto React, Node.js y npm se utilizan habitualmente para instalar dependencias, ejecutar el servidor de desarrollo y construir los archivos de producción. Esto no significa que React necesite Node.js para ejecutarse en el navegador una vez que la aplicación está construida.

6. ¿Cómo instalar Node.js?

Se puede descargar desde su página oficial:

A octubre de 2026, la página oficial ofrece Node.js 24 como versión LTS y Node.js 26 como versión Current. Para comenzar un proyecto de clase o un sistema que se utilizará en producción, normalmente se recomienda la versión LTS compatible con las dependencias del proyecto.

Después de instalarlo, abre la terminal y verifica la instalación:

Bash

```
node --version
npm --version
```

Si ambos comandos muestran números de versión, se podrá comenzar a trabajar.

Conclusión

Node.js amplía las posibilidades de JavaScript al permitir desarrollar el backend, crear servidores y construir APIs. Junto con React, Express y una base de datos, proporciona un entorno adecuado para desarrollar aplicaciones web completas.

Una secuencia recomendable es:

REACT Y COMPONENTES

JavaScript moderno → Node.js y npm → servidor HTTP → Express y APIs REST → conexión con bases de datos → integración con React.

React: componentes, props, state.

En React, los componentes, las props y el state (estado) son tres conceptos fundamentales para construir aplicaciones web modernas. Permiten dividir una interfaz en partes reutilizables, transmitir información entre ellas y actualizar la pantalla cuando cambian los datos.

1. Componentes en React

Un componente es una parte independiente y reutilizable de la interfaz de usuario. Puede representar un botón, un formulario, una tarjeta de estudiante, una tabla de datos o incluso una página completa.

Los componentes suelen definirse como funciones de JavaScript que devuelven elementos de interfaz mediante JSX.

Componente (TarjetaEstudiante)

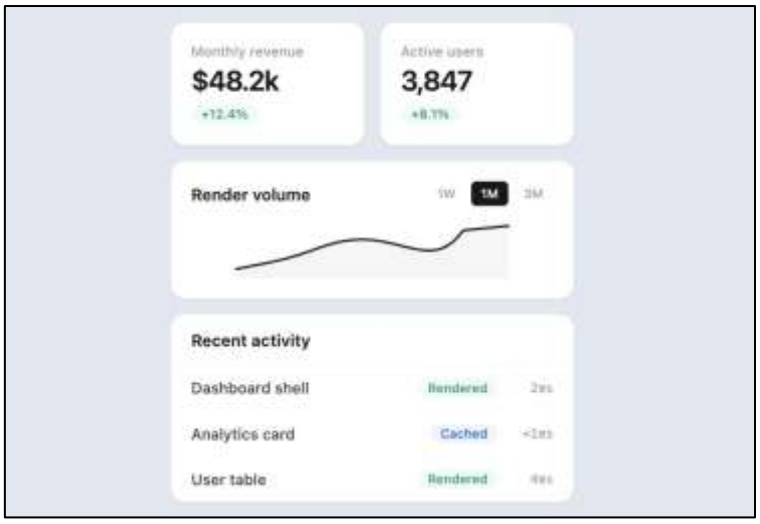
Muestra el nombre, código, carrera y otros datos de un estudiante.



Componente (PanelEstadisticas)

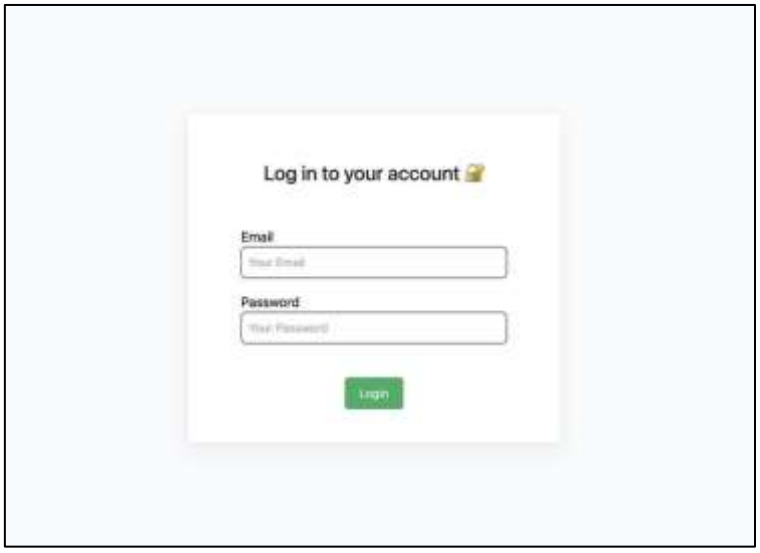
Presenta indicadores como estudiantes matriculados, cursos y promedios.

REACT Y COMPONENTES



Componente (FormularioLogin)

Permite ingresar las credenciales para iniciar sesión.



JavaScript

```
function Saludo() {
  return (
    <h1>Bienvenidos a React</h1>
  );
}

export default Saludo;
```

En este ejemplo:

- function Saludo() declara el componente.
- return indica qué interfaz debe mostrar.
- <h1> es un elemento JSX.

- export default Saludo permite importar el componente desde otro archivo.

Para utilizarlo en otro componente:

JavaScript

```
import Saludo from "./Saludo";

function App() {
  return (
    <div>
      <Saludo />
      <Saludo />
    </div>
  );
}

export default App;
```

La ventaja es que podemos reutilizar Saludo varias veces sin duplicar su implementación.

2. Props: propiedades que recibe un componente

Las props (properties) son datos que un componente recibe desde el componente que lo utiliza. Sirven para personalizar su contenido y comportamiento. Por ejemplo, una tarjeta puede mostrar diferentes estudiantes utilizando el mismo componente.

REACT Y COMPONENTES

JavaScript

```
function TarjetaEstudiante(props) {
  return (
    <div>
      <h2>{props.nombre}</h2>
      <p>Código: {props.codigo}</p>
      <p>Carrera: {props.carrera}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <TarjetaEstudiante
        nombre="Ana Torres"
        codigo="2026001"
        carrera="Ingeniería de Software"
      />
      <TarjetaEstudiante
        nombre="Luis Pérez"
        codigo="2026002"
        carrera="Ingeniería de Sistemas"
      />
    </div>
  );
}
```

En este caso, React utiliza el mismo componente para representar dos estudiantes diferentes.

¿Qué debemos recordar de las props?

- Son la información que recibe un componente.
- Pueden contener cadenas de texto, números, booleanos, objetos, arreglos o funciones.
- Permiten reutilizar un componente con datos diferentes.
- Se consideran de solo lectura: un componente no debe modificar directamente sus propias props.
- Los datos suelen fluir del componente padre al componente hijo.

Una forma más habitual de escribir el componente anterior es mediante la desestructuración de props:

JavaScript

```
function TarjetaEstudiante({ nombre, codigo, carrera }) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>Código: {codigo}</p>
      <p>Carrera: {carrera}</p>
    </div>
  );
}
```

Esta sintaxis permite acceder directamente a cada propiedad sin escribir props.nombre, props.codigo, etc.

3. State: el estado interno de un componente

El state es la información que un componente necesita recordar y que puede cambiar durante la ejecución de la aplicación.

Por ejemplo:

- El número de veces que se presionó un botón.
- El texto escrito en un formulario.
- Si una ventana está abierta o cerrada.
- Los productos agregados a un carrito de compras.
- Si un usuario marcó una tarea como completada.

En los componentes funcionales de React, se utiliza el Hook useState para declarar y actualizar el estado. Cuando el estado cambia, React vuelve a renderizar el componente para reflejar la nueva información en la interfaz.

Ejemplo: contador interactivo

REACT Y COMPONENTES

JavaScript

```
import { useState } from "react";

function Contador() {
  const [contador, setContador] = useState(0);

  function incrementar() {
    setContador(contador + 1);
  }

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={incrementar}>
        Incrementar
      </button>
    </div>
  );
}

export default Contador;
```

¿Cómo funciona?

1. Se declara el estado

`useState(0)` establece el valor inicial del contador en cero.

2. El usuario interactúa

Al presionar el botón, se ejecuta la función `incrementar`.

3. Se actualiza el estado

`setContador` solicita que el valor pase al siguiente número.

4. Se actualiza la interfaz

React vuelve a renderizar el componente y muestra el nuevo contador.

Es importante entender que no basta con modificar una variable normal para que React actualice la interfaz. Para gestionar el estado, debemos utilizar la función de actualización proporcionada por useState. Por ejemplo, esta forma no es apropiada para actualizar el estado:

JavaScript

```
contador = contador + 1;
```

En cambio, debemos utilizar:

JavaScript

```
setContador(contador + 1);
```

Cuando el nuevo estado depende del anterior, también es recomendable emplear la función de actualización:

JavaScript

```
setContador(valorAnterior => valorAnterior + 1);
```

4. Diferencias entre componentes, props y state

Concepto	¿Para qué sirve?	Ejemplo
Componente	Define una parte de la interfaz.	`TarjetaEstudiante`
Props	Recibe datos desde otro componente.	`nombre="Ana"`
State	Conserva datos que pueden cambiar.	`contador = 3`

Una manera sencilla de recordarlos es la siguiente:

- Componente: define qué parte de la interfaz se construye.
- Props: indican qué información recibe.
- State: permite recordar información que puede cambiar con la interacción del usuario.

5. Ejemplo integrador: lista de tareas

En una aplicación de tareas, estos tres conceptos trabajan juntos:

REACT Y COMPONENTES

JavaScript

```
import { useState } from "react";

function Tarea({ texto }) {
  return <li>{texto}</li>;
}

function ListaTareas() {
  const [tareas, setTareas] = useState([
    "Estudiar React",
    "Practicar JavaScript"
  ]);

  function agregarTarea() {
    setTareas([
      ...tareas,
      "Desarrollar una aplicación web"
    ]);
  }
```

```
  return (
    <div>
      <h2>Mis tareas</h2>

      <ul>
        {tareas.map((tarea, indice) => (
          <Tarea key={indice} texto={tarea} />
        ))}
      </ul>

      <button onClick={agregarTarea}>
        Agregar tarea
      </button>
    </div>
  );
}

export default ListaTareas;
```

En este ejemplo:

a) ListaTareas y Tarea son componentes.
b) texto es una prop que recibe cada componente Tarea.
c) tareas es el estado que conserva la lista actual.
d) setTareas actualiza el estado al agregar una nueva tarea.
e) map() genera un componente Tarea por cada elemento de la lista.

f) key permite a React identificar cada elemento de la lista; en una aplicación real conviene utilizar identificadores únicos y estables.

El operador ...tareas crea un nuevo arreglo a partir de las tareas existentes antes de agregar la nueva. En React, es preferible actualizar los arreglos y objetos de estado de forma inmutable, en lugar de modificarlos directamente.

Idea clave: las props permiten comunicar datos entre componentes, mientras que el state permite gestionar información cambiante. Comprender cómo se combinan ambos conceptos es esencial para desarrollar formularios, listas, paneles de control y otras aplicaciones web interactivas con React.

Eventos

En React, los eventos son acciones o interacciones que ocurren en una aplicación web y que permiten ejecutar código como respuesta a lo que hace el usuario. Por ejemplo, hacer clic en un botón, escribir en un campo de texto, enviar un formulario o mover el puntero sobre un elemento.

Los eventos son fundamentales para crear aplicaciones interactivas, ya que permiten que los componentes respondan a las acciones del usuario y actualicen su estado y su interfaz.

1. Tipos de eventos en React

Estos son algunos de los eventos más utilizados en aplicaciones web:

Evento	¿Cuándo ocurre?	Uso habitual
`onClick`	Cuando se hace clic en un elemento.	Ejecutar una acción o pulsar un botón.
`onChange`	Cuando cambia el valor de un campo.	Capturar datos de formularios.
`onSubmit`	Cuando se envía un formulario.	Validar o registrar información.
`onMouseEnter`	Cuando el puntero entra en un elemento.	Mostrar información adicional.
`onMouseLeave`	Cuando el puntero sale de un elemento.	Ocultar información adicional.
`onKeyDown`	Cuando se presiona una tecla.	Detectar teclas como Enter o Escape.
`onFocus`	Cuando un campo recibe el foco.	Resaltar el campo activo.
`onBlur`	Cuando un campo pierde el foco.	Validar un campo después de editarlo.

En JSX, los nombres de los eventos suelen escribirse en camelCase, como onClick y onChange, y reciben una función que se ejecutará cuando ocurra el evento.

2. ¿Cómo se manejan los eventos en React?

Para responder a un evento, normalmente seguimos tres pasos:

Paso 1. Definir una función

Escribimos las instrucciones que deben ejecutarse.

Paso 2. Asociar la función al evento

Usamos una propiedad como `onClick` en el elemento JSX.

Paso 3. Ejecutar la acción

Cuando ocurre el evento, React llama a la función correspondiente.

Ejemplo 1: evento onClick

El siguiente componente muestra un mensaje cuando el usuario presiona un botón.

JavaScript

```
function BotonSaludo() {
  function manejarClick() {
    alert("¡Bienvenido a React!");
  }

  return (
    <button onClick={manejarClick}>
      Mostrar saludo
    </button>
  );
}

export default BotonSaludo;
```

Explicación:

- manejarClick() contiene la acción que se ejecutará.
- onClick={manejarClick} asocia la función al evento de clic.
- El mensaje aparece cuando el usuario pulsa el botón.

Un error frecuente es escribir onClick={manejarClick()}. Esto ejecuta la función durante el renderizado, en lugar de esperar al clic. Debemos pasar la función sin llamarla: onClick={manejarClick}.

Ejemplo 2: evento onChange en un campo de texto

Este evento permite capturar lo que el usuario escribe en un formulario.

JavaScript

```
import { useState } from "react";

function FormularioNombre() {
  const [nombre, setNombre] = useState("");

  function manejarCambio(evento) {
    setNombre(evento.target.value);
  }

  return (
    <div>
      <label htmlFor="nombre">
        Escribe tu nombre:
      </label>

      <input
        id="nombre"
        type="text"
        value={nombre}
        onChange={manejarCambio}
      />

      <p>Tu nombre es: {nombre}</p>
    </div>
  );
}

export default FormularioNombre;
```

En este ejemplo:

- onChange detecta los cambios del campo.
- evento.target.value obtiene el texto introducido.
- setNombre() actualiza el estado.
- React vuelve a renderizar el componente y muestra el texto actualizado.

Este ejemplo relaciona tres conceptos importantes: eventos, estado (state) y componentes.

3. Evento onSubmit: procesar formularios

Cuando un usuario completa un formulario y presiona el botón de envío, se produce el evento onSubmit. En React, normalmente se utiliza event.preventDefault() para evitar

que el navegador recargue la página al enviar el formulario y poder procesar los datos desde JavaScript.

JavaScript

```
function FormularioRegistro() {
  function manejarEnvio(evento) {
    evento.preventDefault();

    alert("Formulario enviado correctamente");
  }

  return (
    <form onSubmit={manejarEnvio}>
      <input
        type="text"
        placeholder="Nombre del estudiante"
        required
      />

      <button type="submit">
        Registrar
      </button>
    </form>
  );
}

export default FormularioRegistro;
```

En una aplicación real, dentro de manejarEnvio podríamos obtener los datos, validarlos y enviarlos al backend mediante una petición HTTP, por ejemplo con fetch.

4. ¿Qué es el objeto del evento?

Cuando React ejecuta una función manejadora, esta puede recibir un objeto que contiene información sobre el evento. Por convención, suele llamarse event o simplemente e.

Por ejemplo:

JavaScript

```
function manejarTecla(event) {
  console.log(event.key);
}
```

Si se asocia a un campo de entrada mediante onKeyDown, event.key permite conocer la tecla presionada.

Hay dos métodos especialmente importantes:

- event.preventDefault(): evita la acción predeterminada del navegador cuando corresponda, como la recarga al enviar un formulario.
- event.stopPropagation(): impide que el evento continúe propagándose hacia los elementos padres.

No son equivalentes: uno evita una acción predeterminada y el otro detiene la propagación del evento.

Conclusión, los eventos permiten que React responda a las acciones del usuario. Al combinarlos con componentes, props y state, podemos desarrollar aplicaciones web dinámicas, desde formularios sencillos hasta sistemas de gestión académica completos.

Renderizado condicional

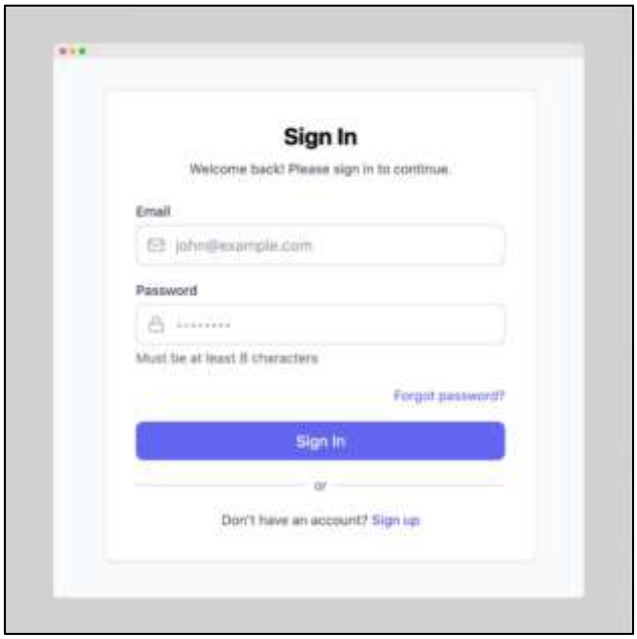
El renderizado condicional en React (conditional rendering) es una técnica que permite mostrar, ocultar o cambiar elementos de una interfaz según se cumpla o no una condición. Por ejemplo, una aplicación puede mostrar una pantalla de inicio de sesión cuando el usuario no está autenticado y un panel de control cuando ya inició sesión. También puede mostrar un mensaje de error cuando un formulario contiene datos incorrectos o indicar si una tarea está completada.

1. ¿Para qué sirve?

El renderizado condicional permite que una aplicación adapte su interfaz a la situación actual.

Inicio de sesión

Mostrar el formulario si el usuario no ha iniciado sesión.



REACT Y COMPONENTES

Panel de control

Mostrar información y opciones después de iniciar sesión.



Validación de formularios

Mostrar mensajes de error cuando los datos no cumplen los requisitos.



2. Formas de implementar el renderizado condicional

En React se pueden utilizar las estructuras condicionales habituales de JavaScript. Las tres formas más importantes para comenzar son if/else, el operador ternario y el operador lógico &&.

A) Utilizando if y else

Esta alternativa permite ejecutar una condición y devolver una interfaz diferente según el resultado.

JavaScript

```
function MensajeUsuario({ estaLogueado }) {
  if (estaLogueado) {
    return <h2>Bienvenido al sistema</h2>;
  } else {
    return <h2>Por favor, inicia sesión</h2>;
  }
}

export default MensajeUsuario;
```

¿Cómo funciona?

- Si estaLogueado es true, se muestra “Bienvenido al sistema”.

- Si estaLogueado es false, se muestra “Por favor, inicia sesión”.

- El componente devuelve una de las dos interfaces, según la condición.

Este método resulta especialmente útil cuando existen varias condiciones o cuando la lógica es relativamente extensa.

B) Utilizando el operador ternario ? :

El operador ternario permite elegir entre dos resultados en una sola expresión.

JavaScript

```
function EstadoMatricula({ matriculado }) {
  return (
    <div>
      {matriculado
        ? <p>El estudiante está matriculado.</p>
        : <p>El estudiante no está matriculado.</p>
      }
    </div>
  );
}
```

Su estructura general es:

JavaScript

```
condicion ? resultadoSiVerdadero : resultadoSiFalso
```

Por tanto, podemos interpretar el código como: “Si el estudiante está matriculado, muestra el primer mensaje; de lo contrario, muestra el segundo”.

C) Utilizando el operador lógico &&

Este operador permite mostrar un elemento solamente cuando una condición es verdadera. Si la condición es falsa, React no muestra ese elemento.

JavaScript

```
function Notificacion({ hayMensajes }) {
  return (
    <div>
      <h2>Bandeja de entrada</h2>

      {hayMensajes && (
        <p>Tienes mensajes nuevos.</p>
      )}
    </div>
  );
}
```

Si hayMensajes es true, aparece la notificación. Si es false, solo se muestra el título.

Precaución: evita expresiones como {cantidad && ... } si cantidad puede ser 0, porque React podría mostrar el número cero. Es más seguro escribir {cantidad > 0 && ... }.

3. ¿Cómo elegir la estructura condicional?

Estructura	Cuándo utilizarla	Ejemplo
`if/else`	Para decidir entre diferentes bloques de interfaz.	Mostrar un panel u otro.
`? :`	Para elegir entre dos alternativas breves.	Mostrar "Aprobado" o "Desaprobado".
`&&`	Para mostrar algo opcional.	Mostrar una alerta solo si existe.
`null`	Para indicar que un componente no debe mostrar contenido.	No mostrar un elemento en determinadas circunstancias.

Por ejemplo, una aplicación de calificaciones puede usar el operador ternario para clasificar una nota:

JavaScript

```
function Resultado({ nota }) {
  return (
    <p>
      {nota >= 11 ? "Aprobado" : "Desaprobado"}
    </p>
  );
}
```

Si nota es 15, se mostrará “Aprobado”; si es 9, aparecerá “Desaprobado”. Los umbrales y reglas deben ajustarse a los criterios académicos que utilice el sistema.

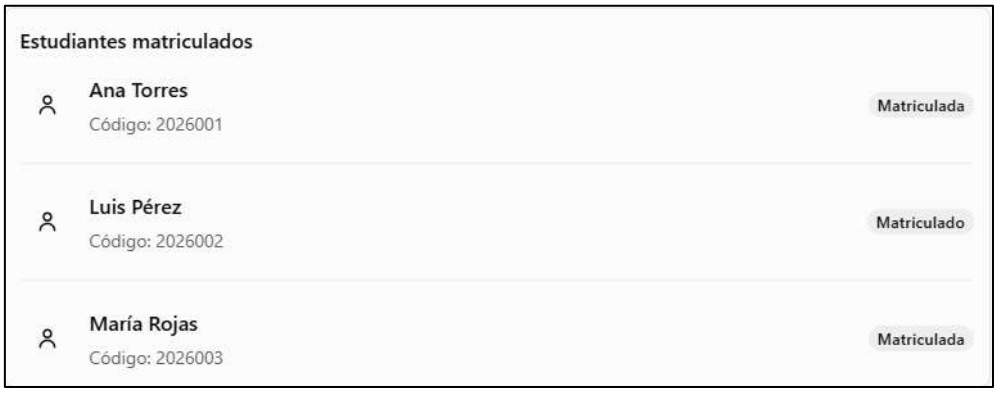
Idea clave: el renderizado condicional permite que una aplicación React no muestre siempre la misma interfaz, sino que adapte los elementos visibles según el estado, las props y las condiciones definidas en el programa.

Listas

En React, las listas permiten mostrar varios elementos de datos de manera dinámica dentro de una interfaz. Por ejemplo, una lista de estudiantes, productos, cursos, tareas o registros de una base de datos. En lugar de escribir manualmente un componente para cada elemento, React permite utilizar arreglos de JavaScript y métodos como map() y filter() para generar, respectivamente, listas de elementos y listas filtradas.

1. ¿Para qué sirven las listas en React?

Imaginemos que estamos desarrollando un sistema de gestión académica.
Necesitamos mostrar los estudiantes matriculados en un curso.



Cada estudiante tiene información diferente, pero la estructura visual es similar. Las listas permiten reutilizar esa estructura y generar tantos elementos como datos tengamos.

2. Utilizar map() para mostrar una lista

El método map() recorre un arreglo y crea un nuevo arreglo con el resultado de aplicar una función a cada elemento. En React se utiliza frecuentemente para generar elementos JSX.

Ejemplo 1: lista de cursos

REACT Y COMPONENTES

JavaScript

```
function ListaCursos() {
  const cursos = [
    "Desarrollo Web",
    "Inteligencia de Negocios",
    "Ingeniería de Software",
    "Base de Datos"
  ];

  return (
    <div>
      <h2>Cursos universitarios</h2>

      <ul>
        {cursos.map((curso, indice) => (
          <li key={curso}>
            {indice + 1}. {curso}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default ListaCursos;
```

¿Cómo funciona?

a) cursos es un arreglo que contiene los nombres de los cursos.
b) map() recorre cada elemento del arreglo.
c) curso representa el nombre del curso actual.
d) indice representa su posición, empezando en cero.
e) crea un elemento de la lista para cada curso.
f) key={curso} proporciona una clave estable en este ejemplo, porque los nombres son únicos.

El resultado será una lista con los cuatro cursos numerados.

3. Mostrar listas de objetos

En las aplicaciones reales, los datos suelen estar organizados en objetos, porque cada registro tiene varias propiedades.

Por ejemplo, cada estudiante puede tener un código, un nombre y una carrera.

REACT Y COMPONENTES

JavaScript

```
function ListaEstudiantes() {
  const estudiantes = [
    {
      id: 101,
      nombre: "Ana Torres",
      carrera: "Ingeniería de Software"
    },
    {
      id: 102,
      nombre: "Luis Pérez",
      carrera: "Ingeniería de Sistemas"
    },
    {
      id: 103,
      nombre: "María Rojas",
      carrera: "Ciencias de la Computación"
    }
  ];
```

```
  return (
    <div>
      <h2>Estudiantes registrados</h2>

      {estudiantes.map((estudiante) => (
        <div key={estudiante.id}>
          <h3>{estudiante.nombre}</h3>
          <p>Carrera: {estudiante.carrera}</p>
          <p>Código: {estudiante.id}</p>
        </div>
      ))}
    </div>
  );
}

export default ListaEstudiantes;
```

Aquí aparecen dos conceptos importantes:

- estudiante.nombre, estudiante.carrera y estudiante.id permiten acceder a las propiedades de cada objeto.
- key={estudiante.id} identifica de manera estable cada elemento de la lista.

¿Por qué es importante la propiedad key?

React necesita identificar los elementos de una lista para reconocer cuáles se agregaron, eliminaron o cambiaron de posición. Por ello, los elementos generados mediante map() deben tener una clave única entre sus elementos hermanos.

Buenas prácticas:

- Utilizar un identificador único procedente de los datos, como el código del estudiante o el ID de la base de datos.
- No generar claves aleatorias en cada renderizado.
- Evitar utilizar el índice del arreglo como clave cuando los elementos puedan insertarse, eliminarse o reordenarse.

Además, key es una propiedad especial de React: si el componente necesita acceder al identificador, debe recibirlo también mediante otra prop, como id.

4. Filtrar listas con filter()

El método filter() permite seleccionar solamente los elementos que cumplen una condición. Por ejemplo, podemos mostrar únicamente a los estudiantes de Ingeniería de Software.

JavaScript

```
function EstudiantesSoftware() {
  const estudiantes = [
    { id: 1, nombre: "Ana", carrera: "Software" },
    { id: 2, nombre: "Luis", carrera: "Sistemas" },
    { id: 3, nombre: "María", carrera: "Software" },
    { id: 4, nombre: "Pedro", carrera: "Computación" }
  ];

  const resultado = estudiantes.filter(
    estudiante => estudiante.carrera === "Software"
  );
```

```
  return (
    <ul>
      {resultado.map(estudiante => (
        <li key={estudiante.id}>
          {estudiante.nombre}
        </li>
      ))}
    </ul>
  );
}
```

El resultado mostrará a Ana y María, porque son quienes cumplen la condición establecida.

La diferencia principal es:

Método	Función
`map()`	Transforma cada elemento y genera un resultado por cada uno.
`filter()`	Selecciona los elementos que cumplen una condición.
`find()`	Busca el primer elemento que cumple una condición.

5. Listas dinámicas con useState

En una aplicación interactiva, una lista puede cambiar cuando el usuario agrega o elimina elementos. Para ello, podemos combinar las listas con el estado de React.

En React, la idea central de esta implementación es mantener el arreglo en el estado y actualizarlo mediante su función de actualización.

Por ejemplo, para agregar un elemento:

JavaScript

```
setTareas([...tareas, nuevaTarea]);
```

Y para eliminar uno:

JavaScript

```
setTareas(tareas.filter(tarea => tarea.id !== id));
```

El ejemplo conserva todas las tareas cuyo identificador sea distinto del que queremos eliminar. Ambos métodos crean nuevos arreglos en lugar de modificar directamente el arreglo anterior.

Idea clave: las listas permiten representar colecciones de datos de forma reutilizable. Al combinar map(), filter(), key y useState, se pueden construir interfaces dinámicas como registros académicos, catálogos de productos, listas de tareas y paneles administrativos.

Organizacion de proyecto frontend

La organización de un proyecto frontend consiste en estructurar sus carpetas, archivos, componentes, estilos, funciones y recursos para que una aplicación web sea fácil de desarrollar, comprender, probar y mantener.

REACT Y COMPONENTES

Una buena organización permite separar las responsabilidades de cada parte de la aplicación y reutilizar componentes sin duplicar código. React no impone una única estructura de carpetas: cada equipo puede elegir la que mejor se adapte al tamaño y las características del proyecto.

1. Estructura básica de un proyecto frontend

Un proyecto creado con React y Vite puede comenzar con una estructura como esta:



Esta es una estructura orientativa, no una regla obligatoria. En proyectos pequeños, algunas carpetas pueden no ser necesarias; a medida que crece la aplicación, podemos incorporar otras.

REACT Y COMPONENTES

2. ¿Cuál es la función de cada carpeta y archivo?

Elemento	Responsabilidad
`public/`	Contiene archivos públicos que se sirven directamente, como el favicon.
`src/`	Contiene el código fuente de la aplicación.
`assets/`	Almacena imágenes, iconos, fuentes y otros recursos importados por el código.
`components/`	Agrupa componentes reutilizables, como botones, tarjetas, tablas y formularios.
`pages/`	Contiene las páginas o vistas principales de la aplicación.
`services/`	Organiza las funciones que se comunican con APIs o servicios externos.
`styles/`	Agrupa hojas de estilo CSS compartidas, si el proyecto lo requiere.
`App.jsx`	Define el componente principal y puede organizar las vistas y el diseño general.
`main.jsx`	Es el punto de entrada que monta la aplicación React en el documento HTML.
`index.html`	Contiene la estructura HTML inicial sobre la que se inicia la aplicación.
`package.json`	Define dependencias, scripts y metadatos del proyecto.
`vite.config.js`	Configura Vite, cuando el proyecto utiliza esta herramienta.

Las carpetas components, pages, services y styles son convenciones útiles, no carpetas que React cree o exija en todos los proyectos.

3. Organización de los componentes

Los componentes son las unidades fundamentales con las que construimos la interfaz en React. Podemos organizarlos según su función y el alcance de su reutilización.

Por ejemplo, en un sistema de gestión académica:



La jerarquía indica qué componentes contienen o utilizan a otros. No significa que todos los datos deban almacenarse en el componente principal: el estado debe situarse donde tenga sentido según los componentes que lo necesitan.

4. Ejemplo práctico de organización

Veamos cómo separar los archivos de un listado de estudiantes.

Archivo 1: src/components/TarjetaEstudiante.jsx

Este componente presenta los datos de un estudiante.

JavaScript

```jsx
function TarjetaEstudiante({ nombre, carrera }) {
  return (
    <article className="tarjeta">
      <h3>{nombre}</h3>
      <p>Carrera: {carrera}</p>
    </article>
  );
}

export default TarjetaEstudiante;
```

Archivo 2: src/pages/Estudiantes.jsx

Esta página importa y reutiliza el componente anterior.

JavaScript

```jsx
import TarjetaEstudiante from
  "../components/TarjetaEstudiante";

function Estudiantes() {
  const estudiantes = [
    { id: 1, nombre: "Ana Torres",
      carrera: "Software" },
    { id: 2, nombre: "Luis Pérez",
      carrera: "Sistemas" }
  ];

  return (
    <main>
      <h1>Estudiantes registrados</h1>

      {estudiantes.map(estudiante => (
        <TarjetaEstudiante
          key={estudiante.id}
          nombre={estudiante.nombre}
          carrera={estudiante.carrera}
```

```
        />
      ))}
    </main>
  );
}

export default Estudiantes;
```

Archivo 3: src/App.jsx

El componente principal incorpora la página.

JavaScript

```
import Estudiantes from "./pages/Estudiantes";

function App() {
  return (
    <div>
      <header>
        <h1>Sistema académico</h1>
      </header>

      <Estudiantes />
    </div>
  );
}

export default App;
```

Archivo 4: src/main.jsx

El punto de entrada monta React en el documento HTML.

JavaScript

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

En el proyecto generado por Vite, index.html debe incluir un elemento con id="root" y cargar el archivo de entrada mediante un script de módulo. La plantilla generada ya proporciona esta configuración.

5. ¿Dónde se coloca la comunicación con el backend?

Cuando el frontend necesita obtener información de un servidor desarrollado, por ejemplo, con Node.js y Express, conviene separar las peticiones HTTP de los componentes visuales.

Una organización posible sería:

```
src/
├── components/
│   └── TarjetaEstudiante.jsx
├── pages/
│   └── Estudiantes.jsx
├── services/
│   └── estudiantesService.js
├── App.jsx
└── main.jsx
```

En src/services/estudiantesService.js:

JavaScript

```
export async function obtenerEstudiantes() {
  const respuesta = await fetch(
    "/api/estudiantes"
  );

  if (!respuesta.ok) {
    throw new Error("No se pudieron obtener los estudiantes");
  }

  return respuesta.json();
}
```

La página puede importar esta función y utilizarla para obtener los datos, normalmente con useEffect y un estado de React para conservar los resultados y gestionar la carga o los errores.

La separación ofrece varias ventajas:

- Los componentes se concentran en presentar información e interacción.
- Los servicios se encargan de las comunicaciones HTTP.

- Los cambios en la API pueden gestionarse sin mezclar toda la lógica con la interfaz.
- Las funciones de acceso a datos pueden reutilizarse y probarse de forma independiente.

6. ¿Cómo organizar un proyecto pequeño y uno grande?

Existen dos enfoques frecuentes:

A) Organización por tipo de archivo

Adecuada para proyectos pequeños y ejercicios iniciales.

```
src/
├── components/
├── pages/
├── services/
├── hooks/
└── App.jsx
```

Es sencilla porque agrupa archivos según su función técnica.

B) Organización por funcionalidad

Útil para aplicaciones con varios módulos y equipos de desarrollo.

```
src/
├── features/
|   ├── estudiantes/
|   |   ├── components/
|   |   ├── services/
|   |   └── pages/
|   ├── cursos/
|   └── matriculas/
├── shared/
|   ├── components/
|   └── hooks/
├── App.jsx
└── main.jsx
```

Cada funcionalidad reúne los archivos que le corresponden, mientras shared/ contiene elementos realmente reutilizados.

No es necesario crear todas estas carpetas desde el primer día. Conviene mantener una estructura sencilla y ampliarla cuando la complejidad del proyecto lo justifique.

REACT Y COMPONENTES

7. Buenas prácticas para organizar el frontend

a) **Separar responsabilidades**: no concentrar toda la aplicación en un único archivo.
b) **Reutilizar componentes**: crear componentes comunes cuando se repiten estructuras o comportamientos.
c) **Nombrar claramente**: utilizar nombres descriptivos, como FormularioMatricula.jsx o ListaCursos.jsx.
d) **Evitar la duplicación**: centralizar las funciones que se utilizan en distintos lugares.
e) **Gestionar el estado apropiadamente**: mantenerlo cerca de los componentes que lo necesitan, compartiéndolo cuando sea necesario.
f) **Separar interfaz y servicios**: evitar mezclar excesivamente la presentación con las peticiones HTTP.
g) **Configurar Git**: utilizar un archivo .gitignore para excluir dependencias instaladas, archivos generados y secretos locales.
h) **No guardar secretos en el frontend**: el código y las variables incluidas en una aplicación cliente pueden ser inspeccionados por los usuarios.

En síntesis: organizar el frontend significa distribuir adecuadamente la interfaz, los componentes, las páginas, los estilos y el acceso a los datos. Una estructura clara facilita que los estudiantes pasen de pequeños ejercicios con React a proyectos web completos, mantenibles y colaborativos.

Reutilización de componentes

La reutilización de componentes en React consiste en crear un componente una sola vez y utilizarlo en diferentes partes de una aplicación, proporcionando datos o propiedades diferentes según las necesidades de cada situación. Es uno de los principios más importantes de React porque permite desarrollar aplicaciones web modulares, consistentes, fáciles de mantener y ampliar. Por ejemplo, en un sistema universitario podemos crear un componente TarjetaEstudiante y utilizarlo para mostrar cientos de estudiantes sin tener que escribir nuevamente el código HTML y JSX para cada uno.

1. ¿Por qué es importante reutilizar componentes?

Imaginemos que estamos desarrollando un sistema de gestión académica que incluye estudiantes, cursos y matrículas.

Tarjeta de estudiante

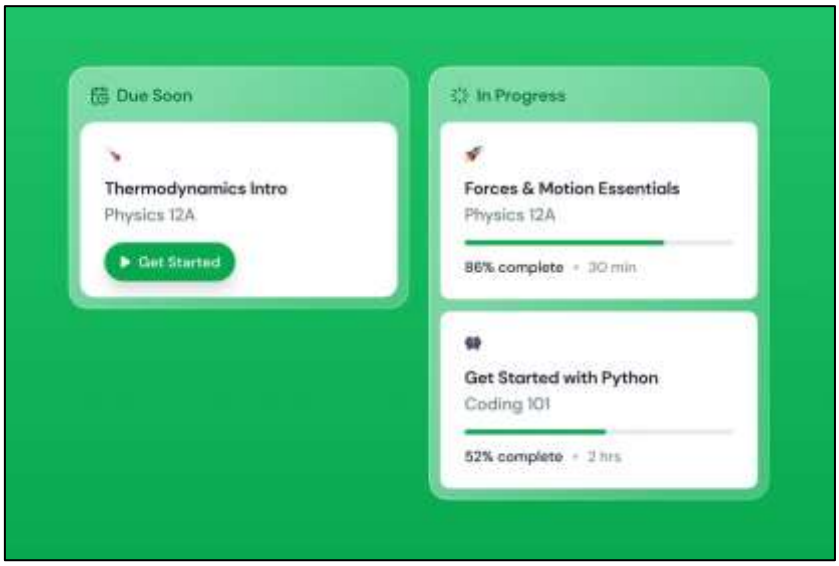
Muestra el nombre, código, carrera y estado de matrícula.

REACT Y COMPONENTES



Tarjeta de curso

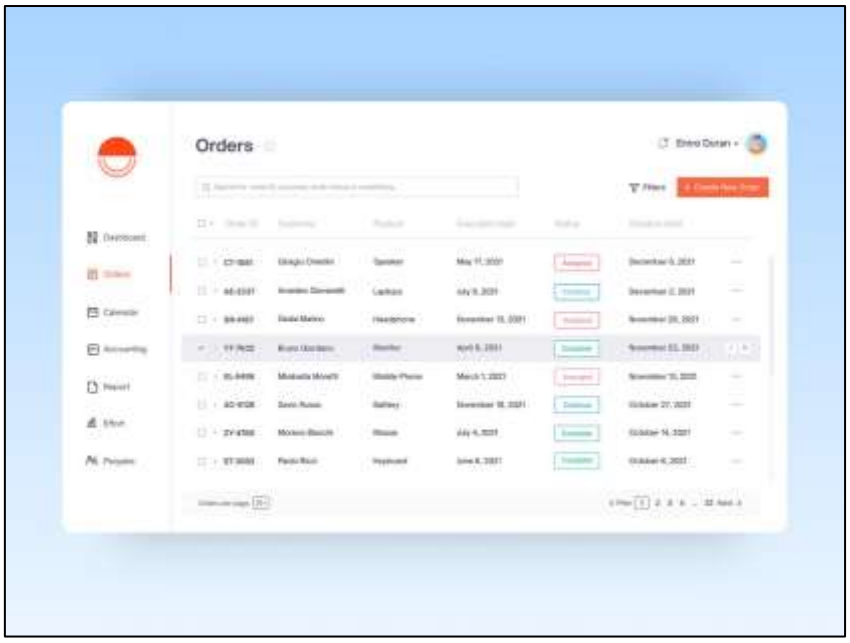
Presenta el nombre del curso, docente y número de créditos.



Botones y formularios reutilizables

Permiten registrar, editar, buscar o eliminar información con un diseño coherente.

REACT Y COMPONENTES



En lugar de repetir el código para cada estudiante, curso o botón, creamos componentes configurables que reciben diferentes datos mediante props.

Las principales ventajas son:

- **Menos duplicación**: se evita escribir repetidamente la misma estructura.
- **Mantenimiento más sencillo**: un cambio en el componente compartido puede reflejarse en todos sus usos.
- **Consistencia visual**: los elementos mantienen un diseño y comportamiento uniformes.
- **Mayor productividad**: se construyen nuevas funcionalidades combinando componentes existentes.
- **Pruebas más fáciles**: los componentes bien delimitados pueden probarse de manera independiente.

2. Ejemplo básico: reutilizar un componente

Supongamos que necesitamos mostrar tres estudiantes. Una alternativa poco recomendable sería escribir cada tarjeta manualmente.

REACT Y COMPONENTES

JavaScript

```
function App() {
  return (
    <div>
      <article>
        <h2>Ana Torres</h2>
        <p>Ingeniería de Software</p>
      </article>

      <article>
        <h2>Luis Pérez</h2>
        <p>Ingeniería de Sistemas</p>
      </article>

      <article>
        <h2>María Rojas</h2>
        <p>Ciencias de la Computación</p>
      </article>
    </div>
  );
}
```

El problema es que repetimos la misma estructura tres veces. Si necesitamos cambiar el diseño, tendríamos que modificar cada tarjeta. Una mejor alternativa es crear un componente reutilizable.

Archivo TarjetaEstudiante.jsx

JavaScript

```
function TarjetaEstudiante({ nombre, carrera }) {
  return (
    <article className="tarjeta">
      <h2>{nombre}</h2>
      <p>{carrera}</p>
    </article>
  );
}

export default TarjetaEstudiante;
```

Archivo App.jsx

JavaScript

```
import TarjetaEstudiante from "./TarjetaEstudiante";

function App() {
  return (
    <div>
      <TarjetaEstudiante
        nombre="Ana Torres"
        carrera="Ingeniería de Software"
      />

      <TarjetaEstudiante
        nombre="Luis Pérez"
        carrera="Ingeniería de Sistemas"
      />

      <TarjetaEstudiante
        nombre="María Rojas"
        carrera="Ciencias de la Computación"
      />
    </div>
  );
}

export default App;
```

Observemos que TarjetaEstudiante se define una sola vez, pero se utiliza tres veces con diferentes valores. La propiedad nombre determina el nombre mostrado, mientras que carrera determina la carrera presentada. La estructura del componente permanece igual.

3. Reutilización mediante listas y map()

Cuando tenemos muchos registros, podemos combinar la reutilización de componentes con las listas de React. Así evitamos escribir manualmente cada tarjeta.

JavaScript

```
import TarjetaEstudiante from "./TarjetaEstudiante";

function ListaEstudiantes() {
  const estudiantes = [
    { id: 1, nombre: "Ana Torres", carrera: "Software" },
    { id: 2, nombre: "Luis Pérez", carrera: "Sistemas" },
    { id: 3, nombre: "María Rojas", carrera: "Computación" }
  ];

  return (
    <section>
      <h1>Estudiantes</h1>

      {estudiantes.map(estudiante => (
        <TarjetaEstudiante
          key={estudiante.id}
          nombre={estudiante.nombre}
          carrera={estudiante.carrera}
        />
      ))}
    </section>
  );
}

export default ListaEstudiantes;
```

Este patrón es especialmente útil cuando los datos provienen de una API o de una base de datos a través del backend. Cada registro se representa con el mismo componente, pero con diferentes props.

4. Reutilización mediante composición y children

No siempre se reutiliza un componente para mostrar datos del mismo tipo. También podemos crear componentes contenedores que reciban otros componentes o contenido mediante la propiedad especial children. Por ejemplo, podemos construir una tarjeta genérica que sirva para estudiantes, cursos o avisos.

REACT Y COMPONENTES

JavaScript

```
function Tarjeta({ titulo, children }) {
  return (
    <section className="tarjeta">
      <h2>{titulo}</h2>
      <div>{children}</div>
    </section>
  );
}

function App() {
  return (
    <div>
      <Tarjeta titulo="Estudiante">
        <p>Nombre: Ana Torres</p>
        <p>Carrera: Ingeniería de Software</p>
      </Tarjeta>

      <Tarjeta titulo="Curso">
        <p>Nombre: Desarrollo Web</p>
        <p>Créditos: 4</p>
      </Tarjeta>
    </div>
  );
}
```

Aquí Tarjeta proporciona una estructura visual común, mientras que children permite insertar contenido diferente dentro de ella. Esta técnica se denomina composición de componentes.

5. ¿Cuándo conviene crear un componente reutilizable?

Una regla práctica es crear un componente cuando:

- Una estructura visual se repite en diferentes partes de la aplicación.
- Una parte de la interfaz tiene una responsabilidad claramente identificable.
- Necesitamos mantener un diseño uniforme.
- El componente tiene suficiente lógica propia como para que separarlo mejore la claridad.
- Queremos probarlo o mantenerlo de manera independiente.

No obstante, no todo elemento debe convertirse en un componente independiente. Crear demasiados componentes pequeños sin una razón clara puede hacer que el proyecto sea más difícil de seguir.

REACT Y COMPONENTES

Buenas prácticas

Práctica	Recomendación
Responsabilidad	Cada componente debe tener un propósito comprensible.
Props	Utilizarlas para personalizar el contenido y el comportamiento.
Estado	Mantenerlo en el componente que lo necesita o en un padre común cuando varios componentes deban compartirlo.
Estilos	Definir estilos coherentes y reutilizables.
Nombres	Utilizar nombres descriptivos, como `BotonGuardar` o `TablaEstudiantes`.
Duplicación	Extraer estructuras repetidas cuando realmente aporte claridad.
Mantenimiento	Evitar modificar directamente las props o el estado; utilizar los mecanismos de React.

Cuando varios componentes necesitan compartir un mismo estado, podemos elevarlo al componente padre común y transmitir los valores y funciones mediante props.

Idea clave: reutilizar componentes no consiste simplemente en escribir menos código; consiste en construir piezas independientes, configurables y coherentes que puedan combinarse para desarrollar aplicaciones cada vez más complejas.